Engineering practices report

Fahrschule Abgefahrn — student app (iOS, Android, web) and admin dashboard · version 1.1.0 · 2026-11-11

This report goes through 97 engineering concepts. Each one says, in one line, what the concept means and then exactly how it is applied in this project, with the files to look at. Tags show where the protection lives: **In code** = implemented in this repository, **Platform** = provided by Supabase / Vercel / Expo, **Process** = a documented practice, **Planned** = not done yet, with the reason.

In code · 73 | Platform · 11 | Process · 9 | Planned · 4

How it fits together

Phones (Expo app) and browsers (static web app on Vercel's CDN) talk over HTTPS to Supabase: an API gateway in front of PostgREST, Auth, Storage, Realtime and eight Deno edge functions. Postgres enforces access with Row Level Security; pg_cron runs maintenance; Resend sends e-mail, Expo sends pushes. Everything stateful is in Postgres, so every other part can scale out or be replaced.

Verification run (2026-11-11)

Unit tests (Vitest)	93 / 93 passed
TypeScript (app)	tsc --noEmit: 0 errors
Edge functions	deno lint + deno check: 8 / 8 clean
Database	20 migrations applied to empty Postgres 17, re-applied idempotently; SQL security suite passed as anon, student, admin, service role
Concurrency	rate limiter: 50 parallel clients × 10 calls → 500 counted, no lost updates
Load (database)	2,000 students / 100k rounds: 10 parallel admins → 52 tps, 192 ms avg, 0 failures
Load (web, production)	400 requests @ 20 parallel → 124 req/s, p50 110 ms, p95 586 ms, 0 errors
Penetration probe (production)	39 / 39 checks passed
Dependencies	0 critical; remaining high advisories reviewed + accepted until 2027-03-31
Bundle budget	JS 3.75 MB raw / 0.95 MB gzip (budget 4.4 / 1.15); no secrets in bundle

Contents

1. 1 · Architecture, scaling & infrastructure (16)
2. 2 · Reliability & resilience (12)
3. 3 · Data & database (9)
4. 4 · Identity & access (15)
5. 5 · Web & input security (16)
6. 6 · Data protection & privacy (8)
7. 7 · Observability & operations (12)
8. 8 · Process, testing & delivery (9)

1 · Architecture, scaling & infrastructure

Load balancing

Platform

Spreading requests over many servers so none is overloaded.

Supabase's gateway balances API, Auth, Storage and edge-function traffic over its instances; Vercel's edge network serves the web app from the nearest location. We run no servers of our own that would need a balancer.

`docs/OPERATIONS.md (Architecture)`

Reverse proxies

In code

A server in front of the app that terminates TLS, routes and filters requests.

Kong (Supabase) and Vercel's edge sit in front of everything. For self-hosting, the Docker image ships nginx as reverse proxy/static server with the same security headers, gzip, health endpoint and SPA routing.

`deploy/nginx.conf, Dockerfile`

API gateways

Platform

One entry point that authenticates, rate-limits and routes API calls.

Supabase's Kong gateway checks the API key and (per function) the JWT before a request reaches PostgREST or a function; per-function JWT verification is declared in config. Our functions add their own rate limits and RBAC behind it.

`supabase/config.toml`

CDNs

In code

Caching static files on servers close to users.

Web build is served from Vercel's CDN. Hashed bundles and assets are cached for a year (immutable); index.html/SPA routes are always revalidated so a new deploy reaches users immediately.

`vercel.json, deploy/nginx.conf`

Horizontal vs vertical scaling

Platform

Horizontal = more machines; vertical = a bigger machine.

Edge functions and the static web app scale horizontally automatically (stateless). Postgres scales vertically (Supabase compute add-ons) and horizontally for reads with replicas. Measured headroom: 2,000 students / 100k quiz rounds → admin dashboard RPC 80 ms; 10 parallel admins 192 ms avg.

`docs/OPERATIONS.md (Scaling)`

Stateless services

In code

Servers keep no session state, so any instance can answer any request.

Every edge function keeps state only in Postgres (rate limits, idempotency keys, webhook dedupe, retry queue). The only in-memory state — circuit breaker and a fallback throttle — is an optimisation that is safe to lose.

`supabase/functions/_shared/serve.ts`

Read replicas

Planned

Copies of the database that serve read-only queries.

Not needed at the current size (one school, ~80 students). Supabase offers replicas on Pro+; the heavy read path (admin performance RPC) is isolated in one function, so it can be pointed at a replica later.

`docs/OPERATIONS.md (Scaling)`

Connection pooling

Platform

Re-using a few database connections for many requests.

The app and functions talk to Postgres over HTTP (PostgREST), which keeps a pool; Supabase's Supavisor pools direct connections. pgbench with 50 parallel clients ran without connection errors.

`scripts/db-load.sh`

Docker

In code

Packaging an app with everything it needs into a portable image.

Multi-stage Dockerfile: Node builds the web export, an unprivileged nginx serves it (healthcheck, STOPSIGNAL SIGQUIT). CI builds the image and smoke-tests health + headers on every push.

`Dockerfile, .github/workflows/ci.yml`

CI/CD

In code

Automatic test (CI) and delivery (CD) on every change.

GitHub Actions: typecheck, 93 unit tests, dependency gate, web export + bundle budget, Deno lint/check of all functions, all migrations + SQL security tests on Postgres 17, Docker build. CD: Vercel deploys main; EAS ships OTA updates and store builds.

`.github/workflows/ci.yml`

Infrastructure as code

In code

Infrastructure described in versioned files instead of clicks.

Database schema, RLS, cron jobs, buckets: supabase/migrations. Function auth: supabase/config.toml. Hosting + headers: vercel.json. Build profiles: eas.json. CI/monitoring: .github/. Self-host: Dockerfile + nginx.conf.

`supabase/, vercel.json, eas.json, .github/`

Message queues

In code

A durable list of jobs processed asynchronously.

Failed invite e-mails form a queue in Postgres (status failed + next_attempt_at, indexed) — the transactional outbox pattern; a worker drains it. A separate broker (RabbitMQ/SQS) would add cost without benefit at this scale.

`migrations/20261111000001, admin-invite-students (retry)`

Background jobs

In code

Work done outside the user's request.

Retry worker re-sends failed invites every 10 min (pg_cron → pg_net → function). Push notifications are sent by a database trigger → push-dispatch. Functions can defer work with EdgeRuntime.waitUntil (ctx.background).

`supabase/scripts/schedule_invite_retry.sql, _shared/serve.ts`

Cron jobs

In code

Tasks on a schedule.

pg_cron: purge rate-limit windows (hourly), webhook dedupe (daily), minimise old import data after 180 days (daily), audit-log retention 2 years (weekly). Uptime check every 15 min in GitHub Actions.

`migrations/20261111000001 §7, .github/workflows/uptime.yml`

Webhooks

In code

A service calls our URL when something happens.

Inbound: Resend delivery events (Svix HMAC signature, 5-min replay window, dedupe by event id), database → push-dispatch (shared secret, constant-time compare, row re-read, once-only), website form → intake-signup (secret).

`resend-webhook, push-dispatch, intake-signup`

Graceful shutdown

In code

Finish running work before a process stops.

Edge runtime: we log the beforeunload event with the number of in-flight requests; background work is registered with waitUntil so the runtime waits for it. nginx in Docker stops with SIGQUIT (finishes open requests).

`_shared/serve.ts, Dockerfile`

2 · Reliability & resilience

Timeouts

In code

Never wait forever on another system.

App: every Supabase request aborts after 30 s (storage 120 s). Functions: outbound calls via fetchWithTimeout (Resend 15 s, Expo 8 s, HIBP 4 s). Database: Supabase statement timeouts per role.

`lib/supabase/client.ts, _shared/http.ts`

Exponential backoff

In code

Wait longer after each failure (1 s, 2 s, 4 s …) plus random jitter.

fetchWithRetry: full-jitter backoff, honours Retry-After, retries only 408/429/5xx/network. Invite retry queue: 5 min·2^n up to 6 h, ±20 % jitter, max 5 attempts. App import retries 2× with backoff. Login lockout grows 15 s → 15 min.

`_shared/http.ts, admin-invite-students, lib/invites.ts, lib/auth/throttle.ts`

Idempotency

In code

Doing the same request twice has the same effect as once.

Imports carry an Idempotency-Key (unique index per admin) → replay returns the first result. Resend batches get an Idempotency-Key so retries never double-send. Webhooks dedupe by event id; pushes once per notification; upserts keyed by e-mail.

`student_imports.idempotency_key, webhook_events`

Circuit breakers

In code

Stop calling a failing service for a while instead of piling up timeouts.

CircuitBreaker (closed → open after 3 failures → half-open after 60 s). While Resend is down, invites are queued for the retry worker instead of waiting on timeouts.

`_shared/http.ts, admin-invite-students`

Error handling

In code

Fail predictably and tell the caller what happened.

HttpError → JSON {error, request_id} with the right status; unknown errors become a generic 500 and are logged, never leaked. Half-created accounts are rolled back (compensating action). App maps every code to German/English text.

`_shared/http.ts (errorResponse), admin-create-student, signup-with-code`

HTTP status codes

In code

The standard meaning of 2xx/4xx/5xx responses.

201 created, 204 preflight, 400 validation, 401 unauthenticated, 403 forbidden, 404, 405 + Allow, 409 conflict, 413 too large, 429 + Retry-After, 500, 502 upstream, 503 not configured — documented per function.

`docs/API.md`

Race conditions

In code

Two operations interleave and corrupt data.

Rate limiter uses one atomic INSERT … ON CONFLICT DO UPDATE (tested: 50 parallel clients, 500 calls → exactly 500 counted). Unique indexes stop duplicate codes/imports; colliding import rows retry one by one; booking uses row locks.

`check_rate_limit, scripts/db-load.sh`

Concurrency

In code

Doing many things at once safely.

Server: Postgres MVCC + unique constraints, independent stateless function instances. Client: double-submit guards (busy refs), in-flight de-duplication of flag requests, Promise.all for parallel loads.

`lib/flags.ts, app/admin/students/import.tsx`

Async/await

In code

Writing asynchronous code that reads top-down.

All I/O is async/await; every promise is awaited or explicitly caught; effects cancel late results with an 'alive' flag so unmounted screens are never updated.

`lib/*, app/*`

Feature flags

In code

Turn features on/off without a release.

app_settings 'feature.*' + feature_flags() RPC; admins flip switches (invite e-mails, list import, performance view) in Administration; functions enforce them server-side (kill switch for mail during an incident).

`lib/flags.ts, app/admin/settings.tsx`

Disaster recovery

Process

Getting back up after losing data or a region.

Daily Supabase backups (PITR optional), schema rebuildable from migrations (proven by CI), off-site dump + storage copy procedure, RPO ≤ 24 h / RTO ≤ 4 h, twice-yearly restore drill.

`docs/OPERATIONS.md (Backups & DR)`

Backups

Platform

Copies of data to restore from.

Automatic daily database backups by Supabase; weekly `supabase db dump` and monthly storage copy recommended into encrypted storage.

`docs/OPERATIONS.md`

3 · Data & database

DB migrations

In code

Versioned, ordered schema changes.

20 SQL migrations, idempotent (if not exists / create or replace), applied in order; CI applies all to an empty Postgres and re-applies the newest to prove they're re-runnable.

`supabase/migrations, scripts/db-test.sh`

Database indexing

In code

Indexes make lookups fast.

Audit found 11 foreign keys without an index (e.g. appointments.slot_id on the booking path) → all added; partial indexes for the retry queue, provider message ids, idempotency keys; audit-log indexes by time/record/actor.

`migrations/20261111000001 §1`

DB normalization

In code

Each fact stored once, related by keys.

Students, packages, topics, classes, enrolments, appointments are separate tables linked by foreign keys (3NF). Deliberate denormalisation: import rows as jsonb snapshot (history), aggregates computed by RPCs instead of stored.

`supabase/migrations/20260804000001_schema.sql`

Soft deletes

In code

Mark rows as deleted instead of removing them.

Students are deactivated (is_active = false) and invites revoked (status) — both reversible. Account deletion is a hard delete on purpose: GDPR/App Store require real erasure.

`profiles.is_active, student_invites.status`

Data validation on server

In code

Never trust the client.

Check constraints (code format, e-mail format, lengths, flag values), guard triggers, and validation in every function (e-mail, UUIDs, sizes, formats, row counts). App checks are only for UX.

`migrations, functions`

Row level security

In code

The database decides row by row who may read/write.

RLS on all 22 tables; students see only their own rows, admins via is_admin(); service-only tables have RLS without policies. 39 live probes + SQL tests as anon/student/admin/service role.

`supabase/tests/10_security.sql`

Time zones

In code

Store instants in UTC, show local time.

Times are timestamptz; school logic (streaks, league week, quiz limit) uses Europe/Berlin; fixed three places that used UTC 'today' (file names, CSV name, reminder ids).

`lib/performance.ts (berlinDay), lib/motivation.ts`

Character encoding

In code

Bytes ↔ text without mojibake.

UTF-8 everywhere; imports detect UTF-8 / UTF-16 / Windows-1252 (German Excel CSV) — verified with umlauts; CSV export has a BOM for Excel; JSON bodies decoded with fatal UTF-8.

`lib/student-import.ts, _shared/http.ts`

Cache invalidation

In code

Making sure caches don't serve stale data.

Web: hashed file names (immutable) + revalidated index.html. Feature flags cached 60 s and invalidated on change. Profile cache cleared on sign-out; screens reload on focus.

`vercel.json, lib/flags.ts, lib/user-context.tsx`

4 · Identity & access

Authentication

In code

Proving who someone is.

Supabase Auth (email + password). Functions verify the JWT with auth.getUser() (signature, expiry, revoked sessions). Biometric app lock on the device as a second layer.

`_shared/serve.ts (authenticate), lib/biometric.ts`

Authorization

In code

Deciding what someone may do.

Database: RLS + guard triggers. Functions: requireAdmin() before any service-role action. UI hides admin areas, but security never depends on the UI.

`_shared/serve.ts (requireAdmin)`

RBAC

In code

Permissions attached to roles.

Roles student/admin in profiles.role; is_admin() used by RLS, RPCs and functions; the last active admin can't be demoted; students can't change their own role.

`lib/rbac/permissions.ts, guard_profile_privileges`

Principle of least privilege

In code

Give each part only what it needs.

App holds only the publishable key; service role only inside functions after authorisation; SECURITY DEFINER functions have fixed search_path and EXECUTE revoked from public; rate limiter callable only by service_role; audit log append-only even for service_role; nginx runs unprivileged.

`migrations, Dockerfile`

Session management

In code

How long-lived sign-ins are created, kept and ended.

Short-lived access tokens + refresh tokens (auto refresh); sign-out revokes server-side (fallback local); the device's push token is now removed on sign-out; recovery sessions end if the user leaves without setting a password.

`lib/user-context.tsx, app/reset-password.tsx`

Token expiration and refresh

Platform

Access tokens expire quickly and are renewed.

Supabase JWT access tokens expire (1 h default) and supabase-js refreshes them in the background (autoRefreshToken); revoked refresh tokens stop the session.

`lib/supabase/client.ts`

Password reset flows

In code

Safe 'forgot password'.

Recovery mail → allow-listed redirect → reset screen. Hardened: a password can only be set right after a recovery link (not on any unlocked signed-in device); new password must pass policy + HIBP; leaving without saving signs out.

`app/reset-password.tsx, lib/auth/recovery.ts`

Brute force protection

In code

Slowing down password/code guessing.

Sign-up codes: 10 tries / 10 min per IP + 5 / h per e-mail (shared limiter) and identical answers for all wrong codes; login: Supabase Auth rate limits + app backoff after 3 wrong passwords.

`signup-with-code, lib/auth/throttle.ts`

Account lockout

In code

Temporarily blocking after repeated failures.

Soft lockout per e-mail in the app (15 s doubling to 15 min). A hard server-side lockout would let attackers lock real students out on purpose; Supabase's IP-based limits are used instead.

`lib/auth/throttle.ts`

Magic links

Planned

Sign in by clicking a link in an e-mail.

Not offered: students sign in many times a day on their phone; mail round-trips hurt that. Supabase supports it (signInWithOtp) if wanted later; the recovery mail already uses the same signed-link mechanism.

`lib/supabase/client.ts`

Pass keys — Planned

Phishing-resistant sign-in with device keys (WebAuthn).

Planned with the next native build (needs associated domains). Today the biometric app lock gives a Face ID / fingerprint gate on the device.

`lib/biometric.ts`

SSO — Planned

One login across services (Google, Microsoft, SAML).

Not needed: students are private individuals invited by the school. Supabase Auth supports OAuth/SAML if the school adopts a workspace.

–

API keys — In code

Secrets that identify a calling application.

Only the publishable key ships in the app (safe by design — RLS protects data). Server keys (service role, Resend) exist only as function secrets; CI blocks builds that contain them.

`scripts/check-bundle.mjs`

IDOR — In code

Insecure direct object reference: reading someone else's record by changing an id.

RLS on every table; delete-account takes the user from the JWT, never the body; tests try to read/update another student's profile and topic stats (blocked).

`supabase/tests/10_security.sql`

Mass assignment — In code

Sending extra fields to change things you shouldn't.

Found and fixed: students could change their own e-mail (used to match accounts!), streak, licence class, topic, created date. Guard trigger now keeps these; functions copy only whitelisted fields.

`guard_profile_privileges (migration 20261111000001 §2)`

5 · Web & input security

Input validations — In code

Check every input for type, length and format.

Server: e-mail regex + length, UUIDs, code patterns, file formats, path regex, row/body limits, JSON object shape, fatal UTF-8. Client mirrors the rules for instant feedback.

`_shared/http.ts (readJson), _shared/invites.ts`

SQL injection — In code

Smuggling SQL through input.

No string-built SQL: PostgREST and supabase-js parameterise; PL/pgSQL uses parameters; dynamic SQL only with format('%I'). Probe: "x' or '1'='1" filter returns nothing; malformed order clause rejected (400).

`scripts/security-probe.mjs`

XSS

In code

Injecting script into pages.

React escapes all output; no innerHTML/dangerouslySetInnerHTML/eval anywhere; invite e-mails HTML-escape names (tested with <img onerror>); CSP script-src 'self' blocks injected scripts.

`_shared/invites.ts (escapeHtml), vercel.json`

Content security policies

In code

Browser allow-list of where scripts, styles and connections may come from.

Strict CSP on the web app: default-src 'self', script-src 'self', object-src 'none', frame-ancestors 'none', base-uri/form-action 'self', connect-src only Supabase/Expo/HIBP. Functions answer with default-src 'none'.

`vercel.json, _shared/http.ts`

Security headers

In code

HTTP headers that switch on browser protections.

HSTS (2 y, preload), X-Frame-Options DENY, nosniff, Referrer-Policy, Permissions-Policy, COOP, CORP, X-Permitted-Cross-Domain-Policies; the uptime job fails if any disappears.

`vercel.json, .github/workflows/uptime.yml`

Clickjacking

In code

Hiding the app in an invisible frame to trick clicks.

frame-ancestors 'none' + X-Frame-Options DENY on web and functions (verified live).

`vercel.json`

Cookie flags (HttpOnly, Secure, SameSite)

In code

Cookie attributes that stop theft and CSRF.

The app sets no auth cookies — tokens are sent as Authorization headers, which also rules out CSRF. Any cookie we add must be HttpOnly; Secure; SameSite=Lax. Token theft via XSS is mitigated by the strict CSP.

`lib/supabase/client.ts`

SSRF

In code

Tricking the server into calling internal URLs.

Functions only call fixed hosts (api.resend.com, exp.host, api.pwnedpasswords.com); no URL ever comes from input.

`supabase/functions/*`

Open redirects

In code

Using our domain to forward victims to an attacker.

No redirect/next/returnTo parameters are read; post-login routes are in-memory constants; recovery redirects are allow-listed in Supabase. Probe confirms parameters are ignored.

`lib/auth/flow.ts, scripts/security-probe.mjs`

Path traversal

In code

Using ../ to reach files outside the intended folder.

Stored file names are generated; the extension now comes from the validated MIME type (fixed); import paths must match ^imports/… without '..'; storage policies scope folders by user id.

`lib/documents.ts (safeDocumentExt), admin-invite-students`

Command injection

In code

Getting the server to run shell commands.

No shell, eval or child processes anywhere in the runtime code (functions run in Deno isolates without subprocess permission).

–

File upload security

In code

Accepting files without accepting attacks.

Private buckets, size caps (10/25 MB) and MIME allow-lists enforced by Storage, admin-only writes, generated names, zip-bomb cap (60 MB unpacked) for xlsx/ods, old .xls refused, CSV formula injection neutralised.

`lib/student-import.ts, migrations (buckets)`

Rate limiting

In code

Capping how often a client may call.

Shared Postgres limiter (fixed window, atomic) on sign-up, imports, sends, creates, deletes, intake; 429 + Retry-After; Supabase Auth limits logins; per-invite 60 s resend cooldown.

`check_rate_limit, _shared/serve.ts`

Environment limiting

In code

Keeping dev, preview and production apart.

EAS build profiles and environments (development/preview/production) with their own variables; Vercel preview deployments per pull request; CI uses placeholder keys; destructive test scripts refuse to run without explicit env vars.

`eas.json, .github/workflows/ci.yml`

HTTPS / TLS

Platform

Encrypted transport.

All endpoints HTTPS only; HTTP is redirected (308, verified); HSTS preload; iOS App Transport Security and Android cleartext-off by default.

`vercel.json`

Penetration testing

In code

Attacking your own system to find holes.

Black-box probe (39 checks, read-only, anon key only) against production: all passed. White-box SQL tests as each role. Internal audit found and fixed mass assignment, reset-password re-auth gap, leaked error messages, non-constant-time secret checks.

`scripts/security-probe.mjs, supabase/tests/`

6 · Data protection & privacy

Secrets management

In code

Keeping keys out of code and logs.

Function secrets / Vault / EAS env / GitHub secrets only; .env and *.local.sql git-ignored; logger redacts passwords, tokens, keys and masks e-mails; CI scans the web bundle for server keys.

`_shared/http.ts (redact), scripts/check-bundle.mjs`

Secret rotation

Process

Replacing keys regularly or after a leak.

Runbook for Resend, webhook/cron/intake secrets, JWT keys and the school sign-up code; webhook verifier accepts several signatures during a switch. The original default sign-up code is in public git history and must be changed in production.

`docs/OPERATIONS.md (Secrets)`

Hashing vs encryption

Platform

Hashing is one-way (passwords); encryption is reversible with a key (data).

Passwords are hashed (bcrypt via Supabase Auth) and never stored or logged by us; HIBP sends only 5 chars of a SHA-1 hash; webhooks are checked with HMAC-SHA256; data at rest/in transit is encrypted, not hashed.

`signup-with-code`

Salting

Platform

A random value per password so equal passwords hash differently.

bcrypt stores a unique salt inside every hash (Supabase Auth); we never handle raw hashes.

–

Encryption at rest

Platform

Stored data is encrypted on disk.

Supabase encrypts database disks, backups and storage (AES-256); secrets in Vault are additionally encrypted. Planned: move the phone session from AsyncStorage to the Keychain/Keystore (expo-secure-store) with the next native build.

`docs/OPERATIONS.md`

Encryption in transit

Platform

Data encrypted on the wire.

TLS for every app ↔ server and server ↔ provider call (Resend, Expo, HIBP); HSTS prevents downgrade.

–

GDPR / data privacy

In code

Lawful, minimal, deletable personal data.

Self-service account deletion now also deletes stored documents; admin delete likewise; import row details minimised after 180 days; audit log stores only needed columns and never the sign-up code; logs mask e-mails; EU-hosted; deletion page live for the stores.

`delete-account, purgeStudentFiles, cron jobs`

Audit logs

In code

Who did what, when — tamper-resistant.

public.audit_log via triggers on profiles (role, active, code, e-mail, class), invites, imports, settings/flags; append-only for every API role (tested); admins can read; 2-year retention.

`migrations/20261111000001 §4`

7 · Observability & operations

Logging

In code

Recording what happened.

Every function logs an access line (method, status, duration) and events; errors with context.

`_shared/serve.ts`

Structured logging

In code

Logs as machine-readable JSON instead of free text.

One JSON object per line: ts, level, fn, request_id, msg + fields — searchable in the Supabase Logs Explorer; secrets redacted automatically.

`_shared/http.ts (createLogger)`

Distributed tracing

In code

Following one request across services.

The app tags every function call with x-request-id; functions reuse it in all log lines, return it in responses and errors, and pass it on (Resend idempotency keys). Full OpenTelemetry is unnecessary at this size.

`lib/supabase/client.ts, _shared/http.ts`

Monitoring

In code

Watching health and performance.

Supabase reports (CPU, connections, slow queries), Vercel analytics, function logs; uptime workflow checks availability, latency (< 5 s) and headers every 15 min.

`.github/workflows/uptime.yml`

Uptime monitoring

In code

Regular external checks that the service is up.

Scheduled GitHub Action every 15 minutes against the web app and Supabase Auth health.

`.github/workflows/uptime.yml`

Alerting

Process

Telling a human when something breaks.

Failed uptime/CI runs e-mail the owner; Dependabot opens security PRs; usage alerts in Supabase/Vercel/Resend (runbook).

`docs/OPERATIONS.md (Monitoring)`

Cost monitoring — Process

Tracking spend before it surprises you.

Spend caps on, monthly checklist (database size, egress, invocations, bandwidth, e-mails, EAS builds at 80 % of the free plan).

`docs/OPERATIONS.md (Cost)`

Debugging — Process

Finding the cause of a bug.

request_id from the app's error → function log lines; stable error codes; local Postgres harness reproduces database behaviour; simulator + web preview for UI.

`docs/OPERATIONS.md (Incident checklist)`

Profiling — In code

Measuring where time goes.

auto_explain on the admin performance RPC: 80 ms for 2,000 students / 100k rounds, the quiz_attempts index scan is the main cost (29 ms); web bundle measured per build.

`scripts/db-load.sh`

Memory leaks — In code

Memory that is never released.

Audit of every effect: all subscriptions (auth, realtime channels, AppState, notifications, timers) are cleaned up. Bounded in-memory maps (throttles) in functions and app.

`components/*, lib/auth/throttle.ts`

Performance budgets — In code

Hard limits on bundle size/speed.

CI fails if web JS exceeds 4.4 MB raw / 1.15 MB gzip or the export exceeds 16 MB (today 3.75 MB / 0.95 MB / 11 MB).

`scripts/check-bundle.mjs`

Mobile responsiveness — In code

Layouts that work on every screen.

Admin kit switches table ↔ cards at 900 px, grids reflow; student app column capped at 672 pt; checked on iPhone simulator, phone width and desktop.

`components/admin/kit.tsx`

8 · Process, testing & delivery

Automated testing — In code

Tests that run without a human.

93 unit tests (Vitest), SQL security suite, Playwright end-to-end specs, load + race scripts; CI runs them on every push.

`tests/, supabase/tests/, e2e/`

Unit vs integration vs end-to-end tests

In code

Small pieces / pieces together / the whole app like a user.

Unit: parsing, planning, retries, breaker, webhooks, throttle. Integration: all migrations + RLS on real Postgres as each role. E2E: Playwright against the web build (login, workflows, layout).

`tests/, supabase/tests/, e2e/`

Load testing

In code

Measuring behaviour under heavy traffic.

Web: 400 requests / 20 parallel → 124 req/s, p50 110 ms, p95 586 ms, 0 errors. Database: 10 parallel admin dashboards on 2,000 students → 52 tps, 192 ms avg.

`scripts/load-test.mjs, scripts/db-load.sh`

Code review

Process

A second person checks every change.

Pull requests with a review checklist (RLS, server validation, secrets, idempotent migrations, i18n, phone + desktop), CODEOWNERS for database and server code, CI must be green.

`.github/pull_request_template.md, .github/CODEOWNERS`

Semantic versioning

Process

MAJOR.MINOR.PATCH communicates the kind of change.

Codebase 1.1.0 (new features, compatible); store version follows native builds; CHANGELOG.md per release.

`CHANGELOG.md, docs/OPERATIONS.md`

API versioning

In code

Changing an API without breaking old clients.

X-API-Version header on every response, unknown majors refused; additive changes only within v1; old store builds keep working because responses only gain fields.

`_shared/http.ts (checkApiVersion)`

API documentation

Process

How to call the API.

Every function documented: auth, request, responses, error codes, limits, idempotency; header comment in each function repeats its contract.

`docs/API.md`

Dependency vulnerabilities

In code

Known holes in libraries.

npm audit fix removed the critical and most high issues; CI gate fails on new high/critical advisories; 3 remaining ones need an Expo SDK upgrade and are accepted with reasons and an expiry date; Dependabot weekly.

`scripts/audit.mjs, security/audit-allowlist.json`

Technical debt Process

Shortcuts that cost later.

Tracked openly: AsyncStorage → SecureStore, custom-scheme recovery links → universal links + PKCE, Expo SDK upgrade (closes accepted advisories), edge functions still use // @ts-nocheck for the Deno globals (shared modules are strictly checked).

`this report § Next steps`

Next steps (need you / the next native build)

1. Apply migration 20261111000001_platform_hardening.sql, then deploy all 8 edge functions (including the new resend-webhook).
2. Set secrets: RESEND_WEBHOOK_SECRET (Resend → Webhooks), CRON_SECRET (+ run supabase/scripts/schedule_invite_retry.sql), optional ALLOWED_ORIGINS.
3. Change the school sign-up code in Administration → Anmeldecode (the original default is in public git history).
4. Move the WordPress intake form's secret from ?secret= to the x-intake-secret header, then rotate INTAKE_SECRET.
5. Add the SUPABASE_ANON_KEY repository secret so the uptime job also checks the API.
6. Next native build (1.0.2): expo-secure-store for sessions, universal links + PKCE for recovery, passkeys.
7. Plan the Expo SDK upgrade to close the accepted advisories before 2027-03-31.

Generated by scripts/build-practices-report.py. Runbooks: docs/OPERATIONS.md · API: docs/API.md · Security policy: SECURITY.md · Changes: CHANGELOG.md.